

[Automotive Data Engine](#)

Tasks

Create a task Retrieve a task Retrieve Multiple Tasks Cancel Task Set Task Metadata Update unique_id Delete unique_id Add Task Tag Set Task Tag Delete Task Tag Avoiding Duplicate Tasks

Create a task

A task represents an individual unit of work to be done by a Contributor. There's a 1:1 mapping between a task and the data to be labeled. For example, there'd be 1 task for each image, video, or lidar sequence needing to be labeled.

You specify how the labeling should be done for a given task when making an API call specifying a set of task parameters to the endpoint you'd like to leverage.

Tasks have a type such as "Image Annotation", "Video Annotation", "Lidar Segmentation", or "Document Transcription".

For information on how to create specific task types, you can click on the links below:

- [Image Annotation Task](#)
- [Semantic Segmentation Annotation Task](#)
- [General Video Annotation Task](#)
- [Lidar Cuboid Annotation](#)
- [Lidar Segmentation Annotation](#)
- [Lidar Linking Annotation](#)
- [Lidar Topdown Tasks](#)
- [Text Collection Task](#)



Tasks objects have a metadata parameter. You can use this parameter to attach key-value data to tasks.

Metadata is useful for storing additional, structured information on an object - especially information that can help you ingest the task response or keep track of what content this task corresponds to.

Metadata is not used by Scale (e.g., to affect how the task is done).

Common use-cases for metadata:

- Internal identifiers
- File paths
- Scenario / Run / Case identifiers
- Environment details (time of day, location)
- Sensor information
- Guideline / Taxonomy versions



```
    completed_at : 2016-06-23T09:10:02.798Z ,
    "type": "categorization",
    "status": "completed",
    "instruction": "Would you say this item is big or small?",
    "params": {
      "attachment_type": "text",
      "attachment": "car",
      "categories": [
        "big",
        "small"
      ]
    },
    "callback_url": "http://www.example.com/callback",
    "callback_completed": true,
    "response": {
      "category": "big"
    },
    "metadata": {},
    "audits": [
      {
        "audited_by": "david@company.com",
        "audited_at": "2016-06-24T15:32:03.585Z",
        "audit_time_secs": 120,
        "audit_result": "accepted",
        "audit_source": "customer"
      },
      {
        "audited_by": "auditor@scale.com",
        "audited_at": "2016-06-23T10:01:02.352Z",
        "audit_time_secs": 511,
        "audit_result": "fixed",
        "audit_source": "scale"
      }
    ],
    "tags": ["experiment_1", "owner:david"],
    "unique_id": "product_experiment_dg3d9x83"
  }
```



enhancing your workflow's data-driven capabilities. Explore task details effortlessly through this essential endpoint.

Path Params `object`

`taskId` `string` `required`

Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc"

# Set up the headers for the request
headers = {
    "accept": "application/json" # Specify that we want the response in JSON format
}

# Adding authentication to the GET request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.get(url, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```



```
type : imageannotation ,
"status": "canceled",
"instruction": "Label every object in this image",
"params": {
  "attachment": "https://example.com/image.jpg",
  "geometries": {
    "box": {
      "objects_to_annotate": [
        "vehicle",
        "pedestrian"
      ]
    }
  }
},
"metadata": {
  "key": "value",
  "key2": "value2"
}
}
```

Retrieve Multiple Tasks

This is a paginated endpoint that retrieves a list of your tasks.

The tasks will be returned in descending order based on `created_at` time. All time filters expect an [ISO 8601-formatted string](#), like '2021-04-25' or '2021-04-25T03:14:15-07:00'

The pagination is based on the `limit` and `next_token` parameters, which determine the page size and the current page we are on. The value of `next_token` is a unique pagination token for each page ([nerdy details if you were curious](#)). Make the call again using the returned token to retrieve the next page.



`end_time` string

The maximum value of `created_at` for tasks to be returned

`completed_after` string

The minimum value of `completed_at` for tasks to be returned

`completed_before` string

The maximum value of `completed_at` for tasks to be returned

`updated_after` array of objects

The minimum value of `updated_at` for tasks to be returned

`updated_before` string

The maximum value of `updated_at` for tasks to be returned

`status` array of objects

The status of the task - can be: `completed` , `pending` , or `canceled`

`type` string

The type of the task.



`customer_review_status` array of objects

The status of the audit result of the task can be: `accepted` , `fixed` , `commented` , or `rejected` ; multiple status values can be specified at the same time either as a joint string separated by comma (e.g. `customer_review_status=accepted,fixed`), or as an array (e.g. `customer_review_status=accepted&customer_review_status=fixed`)

`batch` string

The name of the batch that the returned tasks must belong to

`limit` string

A number between 1 and 100, the maximum number of results to display per page | optional, default 100

`unique_id` array of objects

The `unique_id` of a task; multiple unique IDs can be specified at the same time either as a joint string separated by comma (e.g. `unique_id=a1,a2`), or as an array (e.g. `unique_id=a1&unique_id=a2`)

`tags` array of objects

The tags of a task; multiple tags can be specified at the same time either as a joint string separated by comma (e.g. `tags=t1,t2`), or as an array (e.g. `tags=t1&tags=t2`)

`include_attachment_url` boolean

Set to true if the returned Task Object should include presigned attachment urls.



Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint with query parameters
url = "https://api.scale.com/v1/tasks?status=completed&type=imageannotation&project=kit

# Set up the headers for the request
headers = {
    "accept": "application/json" # Specify that we want the response in JSON format
}

# Adding authentication to the GET request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.get(url, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```




```

    created_at : 2021-06-23T09:09:34.752Z ,
    "callback_url": "http://www.example.com/callback",
    "type": "imageannotation",
    "status": "canceled",
    "instruction": "Label every object in this image",
    "params": {
      "attachment": "https://example.com/image.jpg",
      "geometries": {
        "box": {
          "objects_to_annotate": [
            "vehicle",
            "pedestrian"
          ]
        }
      }
    },
    "metadata": {
      "key": "value",
      "key2": "value2"
    }
  },
  "total": 220,
  "limit": 100,
  "has_more": true,
  "next_token": "eyJ0YXNrX2lkIjoibkYjgwZTFkYmRkNTMwMDExNDZlMzg5Iiwia3JlYXRlZF9hdCI6Ij"
}

```

Cancel Task

You may only cancel pending tasks, and the endpoint will return a 400 error code if you attempt to cancel a completed task

If the task to be canceled had a unique id, specifying `**clear_unique_id=true**` will remove the unique id. Canceling tasks is idempotent such that calling this endpoint multiple times will still return a 200 success response.



Query Params `object`

`clear_unique_id` `string`

If true, will clear a task's `unique_id`, thus allowing the same unique id to be used in future tasks.

Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint with query parameters
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc/cancel?clear_unique_id=tr

# Set up the headers for the request
headers = {
    "accept": "application/json" # Specify that we want the response in JSON format
}

# Adding authentication to the POST request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.post(url, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```



```
    type : imageannotation ,
    "status": "canceled",
    "instruction": "Label every object in this image",
    "params": {
      "attachment": "https://example.com/image.jpg",
      "geometries": {
        "box": {
          "objects_to_annotate": [
            "vehicle",
            "pedestrian"
          ]
        }
      }
    },
    "metadata": {
      "key": "value",
      "key2": "value2"
    }
  }
```

Set Task Metadata

This endpoint sets the `**metadata**` field on a task.

You may set the `**metadata**` field on any existing task using valid key-value data.

Updating a task's `**metadata**` field is idempotent such that calling this endpoint multiple times will still return a 200 success response.

Path Params `object`

`taskId` `string` `required`



```
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc/setMetadata"

# Set up the headers for the request
headers = {
    "accept": "application/json",      # Specify that we want the response in JSON for
    "content-type": "application/json" # Specify the content type of the request
}

# Define the payload for setting metadata
payload = {
    # Add your metadata here
    # For example: "metadata_key": "metadata_value"
}

# Adding authentication to the POST request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.post(url, headers=headers, json=payload, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```



```
    type : imageannotation ,
    "status": "canceled",
    "instruction": "Label every object in this image",
    "params": {
      "attachment": "https://example.com/image.jpg",
      "geometries": {
        "box": {
          "objects_to_annotate": [
            "vehicle",
            "pedestrian"
          ]
        }
      }
    },
    "metadata": {
      "key": "value",
      "key2": "value2"
    }
  }
```

Update unique_id

Easily enhance task management and data accuracy with the Scale Update Task Unique ID API endpoint. Seamlessly modify and optimize task identifiers, ensuring your task tracking and organization remain precise and efficient. This endpoint empowers you to maintain data integrity and adaptability, offering a streamlined way to manage unique IDs associated with tasks within your workflow. Explore this versatile endpoint to effortlessly tailor task identification according to your evolving needs.



Body Params **object**

Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc/unique_id"

# Define the payload to set the unique ID for the task
payload = {
    "unique_id": "56766ba764ee6c4761f6f9b6015657cc6" # Unique ID to be set
}

# Set up the headers for the request
headers = {
    "accept": "application/json", # Specify that we want the response in JSON for
    "content-type": "application/json" # Specify the content type of the request
}

# Adding authentication to the POST request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.post(url, json=payload, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```



```
type : imageannotation ,
"status": "completed",
"instruction": "Label every object in this image",
"params": {
  "attachment": "https://example.com/image.jpg",
  "geometries": {
    "box": {
      "objects_to_annotate": [
        "vehicle",
        "pedestrian"
      ]
    }
  }
},
"unique_id": "new_unique_id"
"metadata": {}
}
```

Delete unique_id

Enables the secure removal of task identifiers, providing you with enhanced control over your data management processes. You can confidently eliminate obsolete or redundant task unique IDs from your system, maintaining data accuracy and improving workflow organization. Seamlessly integrate this functionality into your task management workflow to ensure your records remain up-to-date and clutter-free. Explore the convenience and flexibility of the Scale Delete Task Unique ID endpoint to optimize your data management practices.



Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc/unique_id"

# Set up the headers for the request
headers = {
    "accept": "application/json" # Specify that we want the response in JSON format
}

# Adding authentication to the DELETE request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.delete(url, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```




```
type : imageannotation ,
"status": "completed",
"instruction": "Label every object in this image",
"params": {
  "attachment": "https://example.com/image.jpg",
  "geometries": {
    "box": {
      "objects_to_annotate": [
        "vehicle",
        "pedestrian"
      ]
    }
  }
},
"metadata": {}
}
```

Add Task Tag

With this endpoint, you can include a list of **tags** to be added to a task. If a **tag** is already associated with the task, it will be ignored to avoid duplication. Please note that setting an empty or null string as a **tag** is not allowed. Ensure to provide valid non-empty strings in the **tags** list to update the task's tags successfully.

Path Params **object**

task_id **string** **required**

ID of the Task to modify



Python Python SDK



```
import requests

# Replace with your actual API key
API_KEY = 'your_api_key_here'

# Define the URL for the API endpoint
url = "https://api.scale.com/v1/task/576ba74eec471ff9b01557cc/tags"

# Set up the headers for the request
headers = {
    "accept": "application/json",          # Specify that we want the response in JSON for
    "content-type": "application/json"    # Specify the content type of the request
}

# Define the payload to set the tags for the task
payload = [
    "tag1",
    "tag2",
    "tag3"
]

# Adding authentication to the PUT request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.put(url, headers=headers, json=payload, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```



Path Params object

task_id string required

ID of the Task to modify

Body Params object

RAW_BODY array of strings

List of tags to add to the task



```
API_KEY = 'your_api_key_here'
TASK_ID = 'task_id_here'

# Define the URL for the API endpoint
url = f"https://api.scale.com/v1/task/{TASK_ID}/tags"

# Set up the headers for the request
headers = {
    "accept": "application/json",          # Specify that we want the response in JSON for
    "content-type": "application/json"    # Specify the content type of the request
}

# Define the payload to set the tags for the task
payload = [
    "tag1",
    "tag2",
    "tag3"
]

# Adding authentication to the POST request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.post(url, headers=headers, json=payload, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```

Delete Task Tag

With this endpoint, you can include a list of `**tags**` to be added to a task. If a `**tag**` is already associated with the task, it will be ignored to avoid duplication. Please note that setting an empty or null string as a `**tag**` is not allowed. Ensure to provide valid non-empty strings in the `**tags**` list to update the task's tags successfully.



Body Params `object`

RAW_BODY `array of strings`

List of tags to add to the task



```
API_KEY = 'your_api_key_here'
TASK_ID = 'task_id_here'

# Define the URL for the API endpoint
url = f"https://api.scale.com/v1/task/{TASK_ID}/tags"

# Set up the headers for the request
headers = {
    "accept": "application/json",          # Specify that we want the response in JSON for
    "content-type": "application/json"    # Specify the content type of the request
}

# Adding authentication to the DELETE request
# The auth parameter requires a tuple with the API key and an empty string
response = requests.delete(url, headers=headers, auth=(API_KEY, ''))

# Print the response text to see the result
print(response.text)
```

Avoiding Duplicate Tasks

Creating duplicate tasks is an issue every team should be mindful to avoid.

Scale AI provides two different mechanisms to prevent duplicate tasks from being created in its task creation endpoints. This allows you to resubmit requests that may have failed in transit or otherwise need to be retried without the risk of creating duplicate tasks.

Option 1: The `unique_id` field

The `unique_id` field is a field available on every task type Scale provides.

Once a `unique_id` has been submitted to Scale, any future task creation requests with the same `unique_id` will fail with a [409 error](#) that also conveniently points to the conflicting task.



Best Practices:

1. `unique_id` should be thought of as your own customizable id for a task. Ideally, this id can be easy to look up based on the data you have available on your side. A good `unique_id` might be the filename being submitted, or other types of metadata like a scene or run id that you use internally.
2. `unique_id` is set globally across all projects and task types. If you'd like to enforce uniqueness only within a project or task type, we recommend simply prepending or appending the project or task type to the unique id itself, problem solved!

Option 2: The Idempotency-Key header

To use this feature, provide a header `Idempotency-Key: \<key\>`. You, the client, are responsible for ensuring the uniqueness of your chosen keys. We recommend using V4 UUIDs.

The results of requests specifying an idempotency key are saved. If we later receive a matching request with the same idempotency key, the saved response will be returned, and no additional task will be created. Note that this behavior holds even when the response is an error. Keys are removed after 24 hours.

If an incoming request has the same idempotency key as a saved request, but the two requests do not match in parameters or the users associated with the two requests are different, we will return a [409 error](#).

In rare situations, we may return a [429 error](#) if two matching requests with identical idempotency keys are made simultaneously. In this case, it is safe to retry.

When would I use this instead of the `**unique_id**` field?

Using the header-based approach is useful in retry logic that catches network or other transient failure modes when you would be immediately retrying the exact same request. Specifically, the feature that allows you to seamlessly get the same task response back if the payload didn't change makes for easier code integrations.



Because Unique Ids are permanently tied to a task, this means if something unexpected happened, it can be hard to recover on your own. We have added two features to help support more robust workflows.

Canceling Tasks

When canceling tasks, there is a `clear_unique_id` query parameter you can specify on the request. See the [Cancel Task endpoint](#) for more details.

Errored Tasks

Sometimes after a task is submitted, it can run into an error, especially in regards to processing attachments.

Everywhere you can specify a unique id, you can also specify `clear_unique_id_on_error: true`. As the param name suggests, if the task reaches an error status, the unique id will automatically be unset, such that you could submit a new task with the same new unique id.



```
# Initialize the ScaleClient with your API key
client = scaleapi.ScaleClient("YOUR_API_KEY_HERE")

# Define the task payload
payload = {
    "project": "your_project_name", # Replace with your project name
    "callback_url": "http://www.example.com/callback",
    "instruction": "Draw a box around each object.",
    "attachment_type": "image",
    "attachment": "http://i.imgur.com/v4cBreD.jpg",
    "unique_id": "unique_task_id_12345", # Replace with a unique identifier for the task
    "geometries": {
        "box": {
            "objects_to_annotate": ["Object"],
            "min_height": 10,
            "min_width": 10,
        }
    },
}

# Attempt to create the task, handling duplicates
try:
    task = client.create_task(TaskType.ImageAnnotation, **payload)
    print(f"Task created successfully: {task.as_dict()}")
except ScaleDuplicateResource as err:
    print(f"Task creation failed: {err.message}")
```

```
{
    "unique_id": "s3://bucket/file.png",
    "instruction": "Do the thing",
    "callback_url": "you@gmail.com",
    ...
}
```



```
curl "https://api.scale.com/v1/task/comparison" \  
  -u "{{ApiKey}}:" \  
  -H "Idempotency-Key: UNIQUE_IDENTIFIER" \  
  -d callback_url="http://www.example.com/callback" \  
  ...
```

[< Batches](#)[Fixless Audits >](#)